Librería AYD

Infraestructura como código





PONDERACIÓN: 5

Horas Aproximadas: 30

Universidad San Carlos de Guatemala

Facultad de ingeniería.

Ingeniería en ciencias y sistemas

Índice

Índice **1**
Competencias **2**
Objetivos **2**
General 2
Específicos 2
Descripción / Enunciado **3**
Requisitos Clave: IaC y Despliegue 3
Documentación Solicitada 4
Entregables **4**
Consideraciones **4**
Cronograma **5**
Valores **5**

Competencias

Aplica y adapta modelos de despliegue en la nube mediante arquitecturas por capas, microservicios y plataformas como Docker y Terraform optimizando la disponibilidad y escalabilidad de los sistemas.

Objetivos

General

Diseñar, construir y desplegar automáticamente una plataforma web para la gestión y consulta del catálogo de una librería, aplicando el patrón arquitectónico de microservicios sobre una arquitectura distribuida en la nube, con el propósito de favorecer la disponibilidad y escalabilidad del sistema mediante el uso de Infraestructura como Código (IaC) con Terraform y la automatización de la configuración y el despliegue con Ansible.

Específicos

- Diseñar y modelar el patrón arquitectónico de microservicios clave (Libros y Ventas) sobre una arquitectura de capas, definiendo la API Gateway como punto de entrada y especificando los contratos de las APIs.
- Establecer el modelo de persistencia para cada microservicio, aprovisionando una base de datos desacoplada y escalable (*managed database*) usando Terraform para el catálogo y para el registro de transacciones.
- Contenerizar todos los microservicios y el frontend usando Docker, y definir un proceso de *build* y *push* a un Registro de Contenedores en la nube.
- Aprovisionar toda la infraestructura de la nube usando Terraform (redes, máquinas virtuales y un Balanceador de Carga) y automatizar la configuración del sistema operativo y el despliegue con Ansible.

Descripción / Enunciado

La presente práctica académica tiene como finalidad aplicar los conocimientos adquiridos sobre patrones arquitectónicos, infraestructura como código, contenerización y automatización de despliegues en la nube.

Para ello, los estudiantes deberán desarrollar una plataforma web sencilla que permita a una librería gestionar y visualizar su catálogo de libros, así como consultar un ranking de los títulos más vendidos, actualizado a partir de las transacciones registradas.

Los usuarios, tanto personal administrativo como clientes, podrán consultar el listado de libros disponibles, incluyendo información sobre su título, autor, precio y existencias, así como visualizar cuáles son los libros más vendidos.

Funcionalidad adicional: la plataforma deberá incorporar un mecanismo de búsqueda y filtrado de libros que permita localizar títulos por nombre o autor, facilitando la navegación y consulta del catálogo.

Para alcanzar estos objetivos, se implementará el patrón arquitectónico de microservicios sobre una arquitectura por capas, separando las responsabilidades en dos servicios principales:

- **Microservicio de Libros:** encargado de gestionar los datos del catálogo, incluyendo título, autor, precio y existencias. Además, deberá proporcionar las operaciones necesarias para consultar, buscar y filtrar libros.
- **Microservicio de Ventas:** encargado de registrar las transacciones de venta, incluyendo el libro vendido, la cantidad y la fecha de la transacción, así como de calcular y exponer el ranking de los libros más vendidos a partir del historial de ventas.

El frontend consumirá las APIs correspondientes para presentar el catálogo de libros, las opciones de búsqueda y el ranking de los títulos más vendidos.

La infraestructura deberá desplegarse de forma automatizada utilizando Terraform y Ansible en una plataforma de nube pública, como Amazon Web Services (AWS) o Google Cloud Platform (GCP), conforme a los requisitos establecidos para la práctica.

Requisitos Clave: IaC y Despliegue

Infraestructura como Código con Terraform: Se utilizará **Terraform** para definir y aprovisionar la infraestructura base. Esto incluye la reserva de **direcciones IP públicas estáticas** para las máquinas virtuales que alojarán el frontend y los dos microservicios. La asignación estática facilita el acceso y las pruebas consistentes en la nube. Además, Terraform se encargará de establecer el modelo de persistencia para cada microservicio mediante el aprovisionamiento de bases de datos gestionadas (*managed databases*) que sean desacopladas y escalables. Esto asegura que la infraestructura completa (computación y datos) sea reproducible, inmutable y esté definida enteramente como código.

Contenerización y Automatización con Docker y Ansible: Todos los componentes (frontend, Microservicio de Libros y Microservicio de Ventas) deberán ser **contenedorizados** usando **Docker**.

Se empleará **Ansible** para automatizar la etapa de configuración (*provisioning* post-creación) y el despliegue final. Esta automatización no solo incluirá la instalación de Docker Engine, sino también la **configuración del sistema operativo** de las máquinas virtuales. Ansible será el responsable de lanzar los contenedores con las **configuraciones de red y seguridad adecuadas**, lo cual es crucial para su correcta integración tanto con el **Balanceador de Carga** (para el tráfico entrante) como con las **bases de datos gestionadas** (para la persistencia). Este proceso asegura que la arquitectura sea completamente portable y se pueda recrear de forma idéntica en cualquier entorno.

Documentación Solicitada

La documentación del proyecto deberá incluir:

1. **Diagrama de Arquitectura de Despliegue:** Los estudiantes deberán elaborar un diagrama UML de despliegue que represente la distribución e interacción del frontend, la API Gateway, los dos microservicios, las bases de datos y los recursos en la nube. Para su elaboración, deberán identificar los componentes del sistema, definir sus conexiones y representar cómo Terraform y Ansible intervienen en el aprovisionamiento y despliegue de la infraestructura.

2. **Manual de usuario:** Un documento que detalle los pasos para interactuar con la plataforma web (frontend). Debe incluir instrucciones claras sobre cómo visualizar el listado de servicios, cómo navegar entre especialidades y cómo consultar los detalles de los servicios, dirigido al usuario final.

Entregables

- Link a repositorio privado en github a través UEDI.

Consideraciones

- Todos los integrantes deben estar presentes el día de la calificación, ya que se validará la participación de los integrantes.
- Agregar al aux de su sección correspondiente desde el primer día después de la publicación del enunciado:

    - Juan Carlos Aragón Sección B (aragonjc)
    - Abdiel Fernando Otzoy Otzín Sección B (AbdielOtzoy)
    - Kevin Hernández Sección A (Kjhg4589)

- La documentación debe ser almacenada en el repositorio utilizando el formato MarkDown
- **Las copias parciales o totales tendrán una nota de 0 y será reportada a la escuela de sistemas.**
- La calificación obtenida puede cambiar si la ingeniera solicita una revisión.
- Todos los integrantes del grupo deberán tener commits sustanciales a lo largo del desarrollo. Utilizar el formato de commits para dejar prueba de su trabajo. Hacer merge de pull requests no cuenta como commit válido.
- El nombre del repositorio debe seguir el formato AYD2_<seccion>_2S2026_P2_G<#>(eg.,AYD2_A_2S2026_P2_G4)

Cronograma

Tarea	Fecha
Asignación de la práctica / Entrega del enunciado	15 de octubre 2026
Fecha de entrega	22 de octubre 2026
Fecha de calificación	24 de octubre 2026

Valores

En el desarrollo de la práctica, se espera que cada estudiante demuestre honestidad académica y profesionalismo. Por lo tanto, se establecen los siguientes principios:

1. **Originalidad del Trabajo**
    - Cada estudiante o equipo debe desarrollar su propio código y/o documentación, aplicando los conocimientos adquiridos en el curso.
2. **Prohibición de Copias y Plagio**
    - Si se detecta la copia total o parcial del código, documentación o cualquier otro entregable, la calificación será de **0 puntos**.
    - Esto incluye la reproducción de código entre compañeros, la reutilización de proyectos de semestres anteriores o el uso de código externo sin la debida referencia.
3. **Uso Responsable de Recursos Externos**
    - El uso de bibliotecas, frameworks y ejemplos de código externos está permitido, siempre y cuando se referencian correctamente y se comprendan plenamente. (Consultar con el catedrático su política)
4. **Revisión y Detección de Plagio**
    - Se podrán utilizar herramientas automatizadas y revisiones manuales para identificar similitudes en los proyectos.
    - En caso de sospecha, el estudiante deberá justificar su código y demostrar su desarrollo individual o en equipo. Si este extremo no es comprobable la calificación será de **0 puntos**.

Al detectarse estos aspectos se informará al catedrático del curso quien realizará las acciones que considere oportunas.